

MODELO DE TRABALHO - AULA 06

Estilos Arquiteturais, Requisitos e Trade-offs

Documento-base para elaboração da atividade do projeto do grupo

Projeto	Edusmart
Grupo	
Integrantes	Rafael camargo, Guilherme miranda, Heitor Rodrigues,Igor, Eduardo Brandao Mourao
Representante	
Data	08/09/2026

Orientação de uso: preencha este modelo com dados e evidências do próprio projeto. Os requisitos já existentes devem ser analisados, validados e, quando necessário, refinados. A simples cópia dos requisitos não caracteriza conclusão da atividade.

Sumário

SUMÁRIO	2
1. Projeto e domínio	2
2. Arquitetura atual	2
2.1 Diagrama da arquitetura atual	2
2.2 Evidências no projeto	2
3. Requisitos funcionais	3
4. Requisitos não funcionais	3
5. Relação RNF x parte afetada	3
6. Problema arquitetural encontrado	3
7. Matriz de decisão arquitetural	3
7.1 Justificativa das notas	4
8. Decisão, trade-off e proposta	4
8.1 Diagrama da arquitetura proposta	4
9. Conclusão do grupo	4
Checklist antes da entrega	4
Planejamento de tempo sugerido	5
Critérios de avaliação	5

1. Projeto e

domínio

O EduSmart é uma plataforma educacional que simula o controle acadêmico de cursos, turmas, professores, alunos, matrículas, avaliações, notas e notificações. O sistema é construído em camadas (model, repository, service e legacy) e foi propositalmente preparado pelo professor com um estado inicial imperfeito, permitindo que o grupo analise, valide e evolua a arquitetura ao longo do semestre. Nesta etapa o projeto ainda não possui integração real com banco de dados nem com sistemas acadêmicos externos todas as integrações externas são simuladas por classes de legado e não há, por enquanto, nenhuma limitação técnica que impeça a evolução ou o escalonamento futuro do projeto.

2. Arquitetura atual

Descreva como o sistema está organizado atualmente. Cite evidências concretas: pastas, arquivos, classes, serviços, repositórios, integrações ou outros elementos.

O sistema está organizado no pacote br.edu.edusmart em quatro áreas de responsabilidade, mais a classe de entrada Main:

- modelo — entidades do domínio acadêmico (Aluno, Professor, Disciplina, Turma, Avaliacao, Nota), classes simples de dados sem regras de validação embutidas.
- repositório — a classe genérica RepositorioEmMemoria, que guarda os objetos em um Map em memória; não existe, até o momento, persistência em banco de dados.
- serviço — a classe ServicoEduSmart, que concentra toda a lógica de negócio: matrícula, criação de avaliações, lançamento de notas, cálculo de média final e encerramento de turma. É também essa classe que aciona diretamente os repositórios e as integrações externas.
- legado — as classes ApiAcademicaLegada, ApiEmailLegada e ApiSmsLegada, que simulam integrações com um sistema acadêmico externo, um serviço de e-mail e um serviço de SMS.
- Main — ponto de entrada da aplicação; monta os objetos de exemplo (aluno, professor, disciplina, turma), executa o cenário de matrícula, avaliação, lançamento de notas e encerramento de turma, e imprime o resultado no console. O fluxo geral é: Main monta o cenário e aciona ServicoEduSmart; o serviço lê e grava dados em RepositorioEmMemoria e manipula as entidades de modelo; ao lançar notas e ao encerrar turmas, o próprio serviço instancia e chama diretamente as classes de legado (ApiAcademicaLegada, ApiEmailLegada, ApiSmsLegada), sem qualquer camada intermediária de abstração.

2.1 Diagrama da arquitetura atual



2.2 Evidências no projeto

Elemento arquitetural	Evidência no projeto	Observação
Model	Pacote br.edusmart.modelo:classes Aluno, Professor, Diciplina, Turma, Avaliacao, Nota	Entidades simples (POJOs), sem validação de estado interna
Repository	RepositorioEmMemoria<T>(Map<String.T>), instanciado 5 vezes em ServicoEduSmart (alunos, professores, disciplinas, turmas, avaliacoes)	Dados existem apenas em memoria; sao pedids ao reiniciar a aplicação
Service	ClasseServicoEduSmat:meto dos matricular, criarAvaliacao, lancarNota, mediaFinal, encerrarTurma	Concentra regra de negocio, acesso a dados e chamadas de integração no mesmo metodo
Legacy	Pacote br.edu.edusmart.legado: ApiAcademicaLegado.enviar Nota, ApiEmailLegada.enviar, ApiSmsLegada.enviarSms	Instanciadas com "new" diretamente dentro de ServicoEduSmart (acoplamento concreto, sem interface).

3. Requisitos funcionais

Selecione pelo menos 2 requisitos funcionais. Confirme onde cada requisito aparece no projeto. Se um requisito existente estiver desatualizado, registre a correção proposta.

RF	Descrição	Evidência no projeto	Validação/ajuste necessário
RF01	O sistema deve permitir a realização da matrícula de um aluno em uma turma ativa	MatriculaService.java	Ajuste necessário: O método de matrícula aceita inscrições mesmo se a turma já atingiu o limite máximo de alunos. É necessário implementar a validação de capacidade da turma antes de salvar
RF02	O sistema deve calcular a média final das avaliações do aluno e emitir uma notificação se ele for aprovado ou reprovado	Aviliacao service.java invocando classes da pasta legacy/	Ajuste necessário: A lógica de cálculo está misturada com chamadas diretas (acopladas) para a classe legacy/Envie a Email.java. O envio de notificações deve ser isolado através de uma interface na camada de negócio

4. Requisitos não funcionais

Selecione pelo menos 3 RNFs relevantes e refine cada um para que seja minimamente verificável. Use métrica, limite, cenário ou forma de validação sempre que possível.

RNF	Qualidade	Formulação verificável	Métrica/limite/cenário	Forma de validação
RNF01	Segurança	Garantir autenticação e controle de acesso conforme o perfil do usuário.	100% de restrição de acesso. Um usuário com perfil "Aluno" deve receber um erro de "Acesso Negado" ao tentar chamar métodos exclusivos do perfil "Professor".	Testes unitários e de integração na camada service simulando diferentes perfis e validação de tokens/permisões via Code Review.

RNF02	Desempenho	As operações comuns devem responder em até 2 segundos em condições normais.	Tempo de resposta menor ou igual a 2 segundos (2000ms) para 95% das requisições (P95) simulando um cenário de uso simultâneo moderado.	Medição de tempo de execução via logs da aplicação ou execução de testes de carga locais/automatizados com ferramentas como K6 ou JMeter.
RNF03	Disponibilidade	O sistema deve manter disponibilidade mínima de 99% durante o período acadêmico.	Uptime mínimo de 99% durante o mês corrente (equivalente a no máximo ~7 horas e 18 minutos de indisponibilidade total acumulada no mês).	Monitoramento do tempo de atividade do servidor da aplicação por meio de ferramentas de telemetria, logs de erro (error rates) ou monitor de status de saúde (Health Checks).

5. Relação RNF x parte afetada

Relacione cada RNF a uma parte concreta da arquitetura e explique o impacto.

RNF	Parte afetada	Por que é afetada?	Evidência
RNF01: Persistência Definitiva de Dados	. Camada de Repositórios (<i>Repository</i>)	Atualmente, os dados acadêmicos de cursos, turmas, alunos e notas são guardados temporariamente em coleções locais na memória. Se a aplicação sofrer uma reinicialização ou queda, todas as informações cadastradas e modificadas pelo usuário serão completamente perdidas.	Uso de estruturas de dados estáticas ou locais (como <i>List</i> ou <i>Map</i>) dentro das classes da pasta <i>repository</i> para simular o armazenamento
RNF02: Desacoplamento do Sistema Legado	Camada de Negócio (<i>Service</i>)	A lógica de controle acadêmico realiza chamadas diretas às classes externas de simulação. Essa amarração faz com que mudanças em regras de terceiros ou atualizações estruturais quebrem diretamente o fluxo principal de negócios da plataforma.	Importação direta e instanciamento de componentes e simuladores localizados na pasta/camada <i>legacy</i> de dentro dos serviços principais.

RNF03: Desempenho e Escalabilidade sob Carga	Componentes de Controle (Controllers / Endpoints)	Como o processamento e a filtragem de dados ocorrem em tempo de execução sobre listas locais em memória ram, consultas volumosas ou um aumento expressivo no volume de alunos ativos degradarão a velocidade de resposta do sistema por falta de paginação e concorrência tratada.	Métodos de busca de dados que retornam coleções inteiras diretamente dos repositórios locais para as classes de serviço sem critérios de paginação ou limites de concorrência.

6. Problema arquitetural encontrado

Seguindo a análise da estrutura em camadas estabelecida no projeto **EduSmart**, o principal problema arquitetural identificado reside no **alto acoplamento e dependência direta da camada de negócios com o estado volátil em memória e as simulações da camada de legado**.

Como o sistema foi inicialmente construído sem uma integração real com um banco de dados relacional ou não-relacional, toda a lógica de persistência e simulação externa (*legacy*) está intrinsecamente amarrada às classes da camada *service*.

Esse cenário evidencia uma severa limitação no critério de **Manutenibilidade e Confiabilidade**. Qualquer evolução futura que demande escalar a plataforma, adicionar segurança real aos dados ou substituir as classes de simulação por APIs acadêmicas de produção exigirá uma reestruturação invasiva no núcleo do sistema, violando as barreiras de isolamento propostas pelo modelo arquitetural modular.

7. Matriz de decisão arquitetural

Compare pelo menos 3 alternativas. Sugestão de escala: 1 = fraco; 3 = adequado; 5 = forte. Multiplique peso x nota e some os resultados.

Alternativas comparadas:

- A) arquitetura monolitica
- B) arquitetura baseada em microserviços
- C) monolito modular

Critério	Peso	A Nota	A P×N	B Nota	B P×N	C Nota	C P×N
Desempenho	5	5	25	3	15	5	25
Segurança	5	3	15	5	25	5	25
Manutenibilidade	4	2	8	5	20	5	20
Disponibilidade/Confiabilidade	4	3	12	5	20	4	16

Complexidade operacional	3	5	15	1	3	4	12
Custo de implementação:	3	5	15	1	3	4	12
TOTAL	24	23	90	20	86	27	110

7.1 Justificativa das notas

A alternativa C — Monólito Modular foi selecionada por apresentar a maior pontuação (110 pontos).

Essa arquitetura permite organizar o EduSmart em módulos bem definidos, como alunos, professores, cursos, turmas, matrículas, avaliações e notas, mantendo o sistema como uma aplicação única. Dessa forma, é possível obter uma boa manutenibilidade e separação de responsabilidades, sem introduzir a complexidade operacional de uma arquitetura baseada em microserviços

A arquitetura monolítica tradicional apresentou bom desempenho e baixa complexidade operacional, porém obteve uma pontuação menor em manutenção, devido ao maior acoplamento entre as funcionalidades.

A arquitetura de microserviços apresentou vantagens em segurança, disponibilidade e escalabilidade, mas sua maior complexidade operacional e custo de implementação tornam essa alternativa menos adequada para o porte atual do EduSmart.

8. Decisão, trade-off e proposta

Decisão: ☒ Manter arquitetura atual ☐ Propor evolução arquitetural

A arquitetura do EduSmart será mantida em seu estilo: aplicação única, monolítica, organizada nas camadas modelo, repositório, serviço e legado, com a classe Main como ponto de entrada. Não haverá migração para microserviços nem introdução de banco de dados nesta etapa.

A decisão é coerente com a matriz da seção 7. A alternativa vencedora (monólito modular) não exige troca de estilo arquitetural, e sim disciplina de módulos dentro do monólito que já existe. As camadas atuais já são a base dessa organização; o que falta são fronteiras internas, já que o ServiçoEduSmart concentra matrícula, avaliação, cálculo de média e encerramento de turma, e ainda instancia ApiAcademicaLegada, ApiEmailLegada e ApiSmsLegada com new, sem interface intermediária.

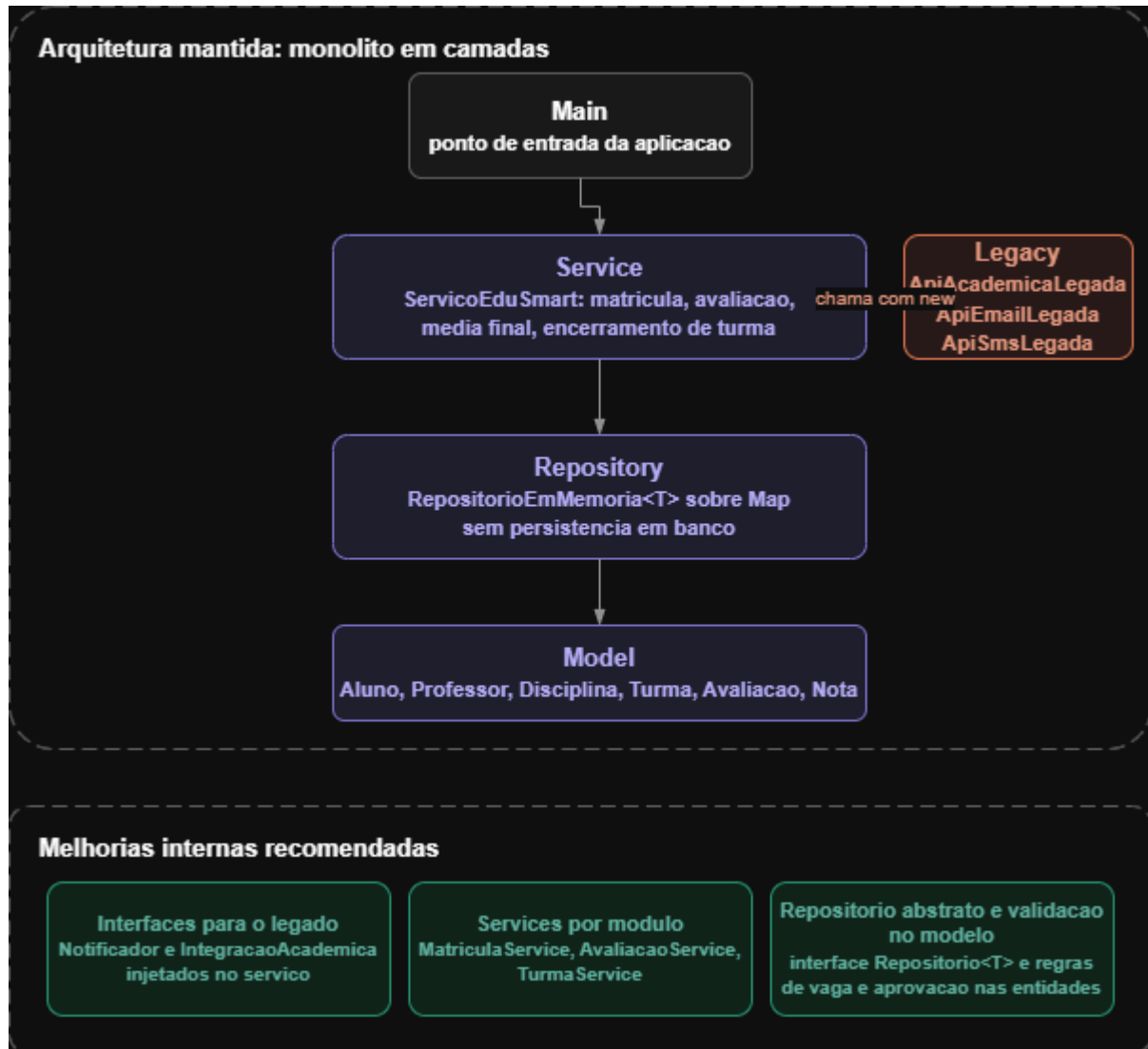
Trade-off aceito: manter a persistência em memória (RepositorioEmMemoria<T> sobre Map<String,T>) e as integrações simuladas pelas classes de legado, abrindo mão de persistência e confiabilidade reais em troca de menor complexidade operacional e menor custo de implementação. São exatamente os dois critérios em que a alternativa de microserviços recebeu a nota mais baixa (1).

Consequências assumidas:

1. Os dados são voláteis. Qualquer reinício da aplicação apaga alunos, turmas, matrículas e notas.

2. Os RNFs de Segurança e Disponibilidade não são verificáveis em ambiente real nesta etapa. Ficam validados apenas por testes na camada service e por revisão de código.
3. O custo de migrar para banco de dados e APIs acadêmicas reais é postergado, mas fica restrito às implementações das novas interfaces, sem alterar a camada de negócio.

8.1 Diagrama da arquitetura proposta



9. Conclusão do grupo

O grupo optou por manter o padrão monolítico em camadas, evoluindo-o internamente para um monólito modular, alternativa de maior pontuação na matriz (110 pontos).

Os requisitos funcionais (RF01 e RF02) mostraram que o problema não está no estilo arquitetural, mas na ausência de fronteiras internas, o que se resolve por refatoração. Os requisitos não funcionais definiram o limite da escolha: Segurança e Disponibilidade dependem

de infraestrutura ainda inexistente no projeto, enquanto Desempenho é atendido no volume atual. Manter o monólito preserva a baixa complexidade operacional e o baixo custo, e a modularização interna recupera a manutenibilidade, que era a maior fraqueza do monólito tradicional.

Checklist antes da entrega

- ☐ SUMÁRIO incluído e atualizado.
- ☐ Arquitetura atual descrita com evidências.
- ☐ Diagrama da arquitetura atual incluído.
- ☐ Pelo menos 2 RF analisados com evidências.
- ☐ Pelo menos 3 RNF refinados e verificáveis.
- ☐ Tabela RNF x parte afetada preenchida.
- ☐ Problema arquitetural real identificado.
- ☐ Três alternativas comparadas na matriz.
- ☐ Pesos, notas, cálculos e justificativas preenchidos.
- ☐ Trade-off e consequência da decisão explicados.
- ☐ Diagrama atual x proposta coerente.
- ☐ Arquivo final exportado em PDF para entrega no Classroom.

Planejamento de tempo sugerido

Etapas	Tempo
Confirmar arquitetura atual e evidências	10 min
Selecionar 2 RF e refinar 3 RNF	12 min
Relacionar RNFs e identificar problema	10 min
Preencher e justificar a matriz	18 min
Decisão, trade-off, proposta e revisão	10 min
Total previsto	60 min

CrITÉRIOS de avaliação

CrITÉrio	Pontuação
Identificação correta da arquitetura atual e evidência no projeto	15,0
Requisitos funcionais coerentes com o domínio do projeto	10,0
Requisitos não funcionais claros, específicos e minimamente verificáveis	20,0
Relação correta entre RNFs e partes afetadas da arquitetura	20,0
Matriz de decisão com critérios, pesos/notas e justificativa	15,0
Identificação de trade-off e consequência da escolha	10,0
Diagrama atual x proposta coerente	10,0
Total	100,0